

Student Handbook

Docker Training

Day 1 · Session 3

Networking & volumes fundamentals

1:00 – 2:30 · 1.5 hours · Hands-on Lab

Prerequisites: Day 1 Sessions 1 & 2 complete — weatherapi:1.0 and weather-ui:1.0 images built

Learning objectives

By the end of this session you will be able to:

- Explain the difference between the default bridge network and a user-defined bridge network
- Create a custom Docker network and connect multiple containers to it
- Use container names as hostnames for service-to-service communication (DNS resolution)
- Demonstrate how network isolation prevents containers on different networks from communicating
- Explain the difference between named volumes, bind mounts, and tmpfs mounts
- Create a named volume and prove that data persists after a container is removed and recreated
- Use a bind mount to enable live hot-reload of Angular source code inside a container
- Back up and restore a named volume using a temporary Alpine container

Key concepts

Network drivers

Bridge (default) Default driver. Containers on the same bridge can communicate. Host traffic is NAT-ed. Does NOT support DNS by container name.	User-defined bridge Custom bridge created with <code>docker network create</code> . Adds automatic DNS — containers reach each other by <code>--name</code> . Always use this for multi-container apps.
Host Container shares the host's network namespace directly. No port mapping needed — but no isolation. Linux only, not available on Docker Desktop.	None Container has zero network access. Used for batch jobs that must be completely isolated from the network.

Storage types

Named volume Managed by Docker. Stored at <code>/var/lib/docker/volumes/</code> . Persists until explicitly deleted. Best choice for databases and stateful services.	Bind mount You specify an absolute host path mapped into the container. Changes on either side are instantly visible on the other. Ideal for dev hot-reload.
tmpfs mount In-memory only — never written to disk. Disappears when the container stops. Use for secrets or ephemeral scratch space.	Port mapping Format: <code>-p HOST_PORT:CONTAINER_PORT</code> . Routes traffic from the host network interface into the container's internal port.

The most important networking rule

The default bridge network does NOT resolve container names as hostnames. Only user-defined (custom) bridges support DNS by container name. This is why you always create a custom network for any multi-container application.

Part A — Container networking

In this part you will: expose the DNS limitation of the default bridge, create a custom network, wire all three services together on it, and verify isolation between networks.

1

Inspect the default network and hit its DNS limitation

Experience the problem before seeing the solution

List existing Docker networks

```
docker network ls
```

Expected output:

NETWORK ID	NAME	DRIVER	SCOPE
alb2c3d4e5f6	bridge	bridge	local
b2c3d4e5f6a7	host	host	local
c3d4e5f6a7b8	none	null	local

Run two containers on the DEFAULT bridge (no --network flag)

```
# Start postgres on the default bridge
docker run -d --name db1 \
  -e POSTGRES_PASSWORD=pass \
  postgres:16

# Start a test container (Alpine with networking tools)
docker run -d --name tester1 alpine sleep 3600
```

Try to reach db1 by container name from tester1

```
docker exec tester1 ping -c 2 db1
```

Expected output:

```
ping: bad address 'db1'

# Name resolution FAILS on the default bridge.
# You must use the IP address — which changes on every container restart.
```

Find the IP and ping by address (the fragile workaround)

```
# Get db1's current IP
docker inspect db1 --format '{{.NetworkSettings.IPAddress}}'
# Example output: 172.17.0.2

# Ping by IP — works but is brittle
```

```
docker exec tester1 ping -c 2 172.17.0.2
```

Expected output:

```
PING 172.17.0.2 (172.17.0.2): 56 data bytes
64 bytes from 172.17.0.2: seq=0 ttl=64 time=0.122 ms
64 bytes from 172.17.0.2: seq=1 ttl=64 time=0.098 ms
```

Why IP-based addressing is fragile

Every time a container is removed and recreated, Docker may assign a different IP address. Hard-coding IPs into connection strings breaks on the next deployment. Container-name DNS on a custom network solves this permanently.

Clean up before the next step

```
docker rm -f db1 tester1
```

2 Create a custom bridge network and wire all three services

Enable DNS resolution so services find each other by name

Create the custom network

```
docker network create appnet
```

Expected output:

```
d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5
```

Inspect the new network

```
docker network inspect appnet
```

Expected output:

```
[
  {
    "Name": "appnet",
    "Driver": "bridge",
    "IPAM": {
      "Config": [{ "Subnet": "172.20.0.0/16", "Gateway": "172.20.0.1" }]
    },
    "Containers": {}
  }
]
```

Start PostgreSQL on appnet

```
docker run -d \
  --name db \
  --network appnet \
  -e POSTGRES_PASSWORD=Training123 \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_DB=traindb \
  postgres:16
```

Start the .NET API on appnet — connection string uses 'db' as hostname

```
docker run -d \
  --name api \
```

```
--network appnet \
-p 8080:8080 \
-e
ConnectionStrings__DefaultConnection="Host=db;Port=5432;Database=traindb;Username=p
ostgres;Password=Training123" \
weatherapi:1.0
```

Start the Angular UI on appnet

```
docker run -d \
--name ui \
--network appnet \
-p 4200:80 \
weather-ui:1.0
```

Verify all three containers are on appnet

```
docker network inspect appnet --format \
'{{range .Containers}}{{.Name}} -> {{.IPv4Address}}{{"\n"}}{{end}}'
```

Expected output:

```
db -> 172.20.0.2/16
api -> 172.20.0.3/16
ui -> 172.20.0.4/16
```

Why Host=db works in the connection string

The `--name db` given to the postgres container becomes a DNS hostname on the appnet network. Docker runs an embedded DNS server at 127.0.0.11 inside each container on a user-defined bridge. Any container on appnet can resolve 'db' to its current IP automatically — even after the container is restarted and gets a new IP.

3

Verify communication and demonstrate network isolation

Confirm DNS works, then show that different networks cannot see each other

Test DNS resolution — ping db by name from the api container

```
docker exec api ping -c 3 db
```

Expected output:

```
PING db (172.20.0.2): 56 data bytes
64 bytes from 172.20.0.2: seq=0 ttl=64 time=0.089 ms
64 bytes from 172.20.0.2: seq=1 ttl=64 time=0.076 ms
64 bytes from 172.20.0.2: seq=2 ttl=64 time=0.081 ms
```

Query PostgreSQL from inside the db container

```
docker exec -it db psql -U postgres traindb -c "SELECT version();" 
```

Expected output:

```

          version
-----
PostgreSQL 16.x on x86_64-pc-linux-gnu, ...
(1 row)
```

Demonstrate isolation — a container on a different network cannot reach db

```
# Start a container on the DEFAULT bridge (no --network flag)
docker run -d --name isolated alpine sleep 3600

# Try to reach db from the isolated container
docker exec isolated ping -c 2 db
```

Expected output:

```
ping: bad address 'db'
# Cannot reach it - different network, complete isolation.
```

A container can belong to multiple networks simultaneously

```
docker network create monitoring
docker network connect monitoring api

# api is now on both appnet AND monitoring
docker inspect api --format \
    '{{range $k,$v := .NetworkSettings.Networks}}{{$k}}{{"\n"}}{{end}}'
```

Expected output:

```
appnet
monitoring
```

Clean up the test containers and extra network

```
docker rm -f isolated
docker network disconnect monitoring api
docker network rm monitoring
```

Networking checkpoint

All three services (ui, api, db) are running on appnet and can reach each other by container name. The .NET connection string uses Host=db and resolves automatically. Containers on other networks cannot reach these services. Move on to volumes.

Part B — Volumes and persistent storage

In this part you will: demonstrate data loss without a volume, add a named volume so PostgreSQL survives container removal, use a bind mount for Angular hot-reload, and back up and restore the volume.

4 Named volumes — persist PostgreSQL data across container restarts

Prove that data survives `docker rm` when stored in a named volume

First, show what happens WITHOUT a volume.

Insert data into the current db container (no volume attached)

```
docker exec -it db psql -U postgres traindb -c "  
  CREATE TABLE IF NOT EXISTS students (id SERIAL, name TEXT);  
  INSERT INTO students (name) VALUES ('Alice'), ('Bob');  
  SELECT * FROM students;  
"
```

Expected output:

```
 id | name  
----+-----  
  1 | Alice  
  2 | Bob  
(2 rows)
```

Remove the container and recreate it — data is lost

```
docker rm -f db  
  
docker run -d \  
  --name db \  
  --network appnet \  
  -e POSTGRES_PASSWORD=Training123 \  
  -e POSTGRES_USER=postgres \  
  -e POSTGRES_DB=traindb \  
  postgres:16  
  
docker exec -it db psql -U postgres traindb -c "SELECT * FROM students;"
```

Expected output:

```
ERROR:  relation "students" does not exist  
# The table is gone — deleted with the container's writable layer.
```

Why this matters

A container's writable layer is stored inside Docker and deleted when the container is removed with `docker rm`. Without a volume, every `docker rm` loses all database data permanently. This is why you must always mount a volume for any stateful service.

Create a named volume

```
docker volume create pgdata

# Inspect to see where Docker stores it on disk
docker volume inspect pgdata
```

Expected output:

```
[
  {
    "Name": "pgdata",
    "Driver": "local",
    "Mountpoint": "/var/lib/docker/volumes/pgdata/_data",
    "Scope": "local"
  }
]
```

Remove the temporary db and recreate it with the named volume

```
docker rm -f db

docker run -d \
  --name db \
  --network appnet \
  -e POSTGRES_PASSWORD=Training123 \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_DB=traindb \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16
```

Insert data again

```
docker exec -it db psql -U postgres traindb -c "
CREATE TABLE students (id SERIAL, name TEXT);
INSERT INTO students (name) VALUES ('Alice'), ('Bob'), ('Charlie');
SELECT * FROM students;
"
```

Expected output:

```
id | name
----+-----
 1 | Alice
 2 | Bob
 3 | Charlie
(3 rows)
```

Remove the container and recreate with the SAME volume — data persists

```
# Remove the container (the volume is NOT affected by docker rm)
docker rm -f db

# Recreate with the same -v pgdata: flag
docker run -d \
  --name db \
  --network appnet \
  -e POSTGRES_PASSWORD=Training123 \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_DB=traindb \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16

# Query — data is still there
docker exec -it db psql -U postgres traindb -c "SELECT * FROM students;"
```


Expected output:

```
id | name
----+-----
1 | Alice
2 | Bob
3 | Charlie
(3 rows)

# Data survived the container being deleted and recreated!
```

What just happened

The named volume pgdata stores data independently of any container. `docker rm` only deletes the container — the volume remains until you explicitly run `docker volume rm pgdata`. You can destroy and recreate the postgres container as many times as you like without losing the database.

5

Bind mounts — live hot-reload for Angular development

Map your local source into the container so edits appear instantly without rebuilding

A bind mount differs from a named volume: you specify the exact host path, and the host and container share the same files in real time.

Understand the syntax difference

```
# Named volume — Docker manages the location
-v pgdata:/var/lib/postgresql/data

# Bind mount — you specify an absolute host path
-v /absolute/path/on/host:/path/in/container
-v $(pwd)/src:/app/src      # $(pwd) = current directory on Mac/Linux
-v ${PWD}/src:/app/src      # PowerShell on Windows
-v %cd%/src:/app/src        # CMD on Windows
```

Stop the production UI container and start a dev server with bind mount

```
docker rm -f ui

# Navigate to your Angular project
cd ~/docker-training/weather-ui

# Run Angular dev server with source bind-mounted
docker run -d \
  --name ui-dev \
  --network appnet \
  -p 4200:4200 \
  -v $(pwd):/app \
  -w /app \
  node:20-alpine \
  sh -c "npm ci && npx ng serve --host 0.0.0.0 --poll 500"
```

Expected output:

```
** Angular Live Development Server is listening on 0.0.0.0:4200 **
```

Test live reload — edit a file on the HOST machine

```
# On the HOST (not inside the container) - edit the app title
sed -i 's/weather-ui/Docker Training App/g' src/app/app.component.html

# Watch the container logs - it detects the change and rebuilds
docker logs -f ui-dev
```

Expected output:

```
Compiling...
Compilation complete. Watching for file changes...
# Angular detected the file change and rebuilt automatically.
```

Open `http://localhost:4200` in your browser. The title should now show 'Docker Training App' without rebuilding the image or restarting the container.

Windows path syntax and --poll

`--poll 500` makes Angular check for file changes every 500ms using polling instead of filesystem events. This is required on Docker Desktop (Mac and Windows) because inotify events from the host filesystem do not always propagate into the container. Without `--poll`, live reload may silently fail.

When to use bind mounts vs named volumes

Bind mounts: source code and config files you edit frequently during development. Named volumes: databases and any data Docker should manage long-term. Never use a bind mount for a database in production — filesystem performance and permission handling are unpredictable across operating systems.

6 Back up and restore a named volume

Use a temporary Alpine container as a data mover — works with any volume

Named volumes are stored inside Docker's managed area on disk. To back them up you use a helper container that can access both the volume and a host directory simultaneously.

Back up the pgdata volume to a tar file on the host

```
# Run a temporary Alpine container with TWO mounts:
#   -v pgdata:/data          mounts the volume at /data
#   -v $(pwd):/backup        mounts the current host directory at /backup
# --rm removes the container automatically when it exits
docker run --rm \
  -v pgdata:/data \
  -v $(pwd):/backup \
  alpine \
  tar czf /backup/pgdata-backup.tar.gz -C /data .

# Verify the backup file exists on the host
ls -lh pgdata-backup.tar.gz
```

Expected output:

```
-rw-r--r-- 1 user user 4.2M Jun 17 10:45 pgdata-backup.tar.gz
```

Simulate a disaster — delete the volume and all data

```
docker rm -f db
```

```
docker volume rm pgdata
```

```
# Confirm the volume is gone
docker volume ls
```

Expected output:

```
DRIVER      VOLUME NAME
# pgdata is not listed — it is deleted
```

Restore the volume from the backup

```
# Step 1: recreate the empty volume
docker volume create pgdata
```

```
# Step 2: extract the backup into the volume
docker run --rm \
  -v pgdata:/data \
  -v $(pwd):/backup \
  alpine \
  tar xzf /backup/pgdata-backup.tar.gz -C /data
```

```
# Step 3: restart postgres with the restored volume
docker run -d \
  --name db \
  --network appnet \
  -e POSTGRES_PASSWORD=Training123 \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_DB=traindb \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16
```

```
# Step 4: verify data came back
docker exec -it db psql -U postgres traindb -c "SELECT * FROM students;"
```

Expected output:

```
id | name
----+-----
 1 | Alice
 2 | Bob
 3 | Charlie
(3 rows)
```

```
# Full restore complete
```

Important: docker rm does NOT remove named volumes

```
# Remove the container
docker rm -f db
```

```
# Volume is still there
docker volume ls
```

Expected output:

```
DRIVER      VOLUME NAME
local       pgdata      <-- still present after docker rm
```

Volume removal commands

`docker volume rm pgdata` — removes one named volume (fails if a container is using it). `docker volume prune` — removes ALL unused volumes. `docker system prune --volumes` — removes

containers, images, networks, AND volumes. Always check what you are removing before running prune commands.

Session outcome

All three services run on a custom network and communicate by name. PostgreSQL data persists in a named volume across container restarts. Angular source hot-reloads from a bind mount without rebuilding the image. Volume data can be backed up and restored using a temporary helper container.

Quick reference

Networking commands

Command	What it does
<code>docker network ls</code>	List all networks
<code>docker network create appnet</code>	Create a user-defined bridge named appnet
<code>docker network inspect appnet</code>	Show full JSON details including connected containers
<code>docker network connect appnet <name></code>	Attach a running container to a network
<code>docker network disconnect appnet <name></code>	Detach a container from a network
<code>docker network rm appnet</code>	Remove a network (must be unused)
<code>docker network prune</code>	Remove all unused networks

Volume commands

Command / Flag	What it does
<code>docker volume create pgdata</code>	Create a named volume
<code>docker volume ls</code>	List all volumes
<code>docker volume inspect pgdata</code>	Show location and metadata
<code>docker volume rm pgdata</code>	Remove a specific volume
<code>docker volume prune</code>	Remove all unused volumes
<code>-v pgdata:/var/lib/postgresql/data</code>	Mount named volume into container
<code>-v \$(pwd)/src:/app/src</code>	Bind mount current directory/src into container
<code>--mount type=tmpfs,target=/tmp</code>	In-memory tmpfs mount (no disk write)

Common mistakes to avoid

Mistake 1 — Using the default bridge and wondering why services can't find each other

The default bridge does not support DNS. If you run `docker run` without `--network`, containers can only communicate by IP. Always create a custom network with `docker network create` and attach all related containers to it.

Mistake 2 — Forgetting `-v` and losing database data on `docker rm`

Without a named volume, all data written inside a container is in its writable layer — deleted

permanently when you run `docker rm`. Always add `-v pgdata:/var/lib/postgresql/data` (or equivalent) to any stateful container.

Mistake 3 — Using a relative path in a bind mount

Docker requires an absolute path on the host side of a bind mount. Use `$(pwd)/src` instead of `./src`, or the command will fail with a cryptic error about volume names.

Mistake 4 — Running `docker volume prune` without checking what is there

`docker volume prune` removes ALL volumes not currently attached to a running container. If your database container is stopped (not running), its volume will be pruned. Run `docker volume ls` first and confirm which volumes are safe to remove.

Coming up — Session 4: Debugging & day-to-day Docker workflow

In the next session you will learn to diagnose problems, monitor resources, and keep your Docker environment clean.

- `docker logs`, `docker stats`, `docker top` — observing what containers are doing
- `docker inspect` with `--format` filters for targeted debugging
- Copying files in and out of containers with `docker cp`
- Setting resource limits: `--memory` and `--cpus`
- System cleanup: `docker system prune` and what each prune command removes

Keep your environment ready

Leave the `db`, `api`, and `ui` containers running on `appnet` with the `pgdata` volume attached. They will be used in Session 4's debugging exercises.